TECMILENIO

Gestión de Proyectos de Inteligencia Artificial

Manual de despliegue en la nube

Docker, GitHub Codespaces y alternativa PaaS en Render

Institución	Universidad Tecmilenio
Materia	Gestión de Proyectos de Inteligencia Artificial
Actividad	Proyecto Fase II - Entregable 3
Proyecto	Evaluación experimental de riesgo crediticio
Alumno	Miguel Ángel Alejandre Pérez
Docente	Luis Ariel Vazquez Piña
Periodo	08 Noviembre de 2026

Repositorio: github.com/miguelalejandreperez-rgb/riesgo-crediticio-fase-II

1. Propósito y alcance

Establecer los requisitos, los pasos de ejecución y las comprobaciones necesarias para poner en funcionamiento el prototipo de evaluación experimental de riesgo crediticio. El sistema integra un formulario web, una API REST y un modelo de clasificación empaquetado en Docker.

Entorno	Estado comprobado
GitHub / rama main	Repositorio actualizado; modelo y documentación versionados.
GitHub Codespaces	Contenedor Docker construido y ejecutado; formulario y API probados.
GitHub Actions	Flujo docker-validation #1 ejecutado correctamente (commit 8726777).
Render (PaaS)	Procedimiento de despliegue definido; no se realizó publicación ni prueba en Render.

2. Arquitectura técnica

Componente	Función principal
Navegador	Muestra el formulario HTML, CSS y JavaScript; envía los datos mediante HTTP/JSON.
FastAPI	Valida las 19 variables y expone GET /api/health, GET /api/modelo y POST /api/predecir.
Modelo	Pipeline HistGradientBoostingClassifier + imputación por mediana; devuelve un puntaje.
Docker	Incluye Python 3.11, bibliotecas, frontend y archivos del modelo.
Render	Opción PaaS para construir la imagen desde GitHub y servir la aplicación por HTTPS.

Flujo de inferencia: navegador → formulario → POST /api/predecir → validación FastAPI → pipeline cargado desde artifacts/modelo_riesgo.joblib → respuesta JSON → presentación de puntuación y alerta.

3. Requisitos previos

- Cuenta de GitHub con acceso al repositorio: https://github.com/miguelalejandreperez-rgb/riesgo-crediticio-fase-II
- Archivos versionados: Dockerfile, app/, requirements-runtime.txt, artifacts/modelo_riesgo.joblib y artifacts/metadata.json.
- Para Codespaces: navegador web y un Codespace del repositorio con Docker-in-Docker habilitado.
- Para Render: cuenta activa, autorización para conectar el repositorio y verificación de cuota y condiciones del plan seleccionado.

El contenedor no entrena modelos durante el inicio. Lee el artefacto serializado y verifica su integridad con los metadatos; el entrenamiento se mantiene como procedimiento separado.

4. Ejecución verificada en GitHub Codespaces

4.1 Abrir el entorno

1. En GitHub, seleccionar Code > Codespaces > Open (o Create codespace on main, si todavía no existe).
2. Abrir la terminal de Codespaces y ubicarse en la carpeta principal del repositorio.
3. Ejecutar los siguientes comandos en orden:

```
python scripts/check_artifacts.py
bash scripts/start_codespaces.sh
bash scripts/smoke_docker.sh
```

El primer comando verifica que el modelo y los metadatos estén presentes y tengan una huella SHA256 consistente. El segundo recupera o crea el contenedor; el tercero comprueba /api/health y una predicción real.

4.2 Acceder al formulario

4. Abrir la pestaña Ports del editor. Confirmar que el puerto 8000 está reenviado y marcado como Private.
5. Seleccionar Open in Browser para abrir la URL vigente del puerto. No utilizar el enlace de una sesión anterior.
6. Completar los 19 campos con valores de prueba y pulsar Calcular puntuación. El panel lateral muestra puntuación, umbral y estado de alerta.
7. Para revisar la API, agregar /docs al final de la dirección y utilizar Swagger UI.

4.3 Recuperación tras detener Codespaces

Codespaces conserva los archivos versionados, pero los contenedores pueden detenerse o desaparecer al recrearse el entorno. Cuando se abra una nueva sesión, ejecutar de nuevo el script de inicio sin entrenar el modelo:

```
bash scripts/start_codespaces.sh
docker ps
bash scripts/smoke_docker.sh
```

Si cambió el modelo, el Dockerfile o el código incluido en la imagen, reconstruirla con bash scripts/start_codespaces.sh --rebuild. Si faltan los artefactos, ejecutar una vez bash scripts/prepare_model.sh y registrar el modelo y sus metadatos en GitHub.

5. Despliegue propuesto en Render

Render proporciona un servicio Web Service capaz de construir una imagen desde un Dockerfile alojado en GitHub. Se propone utilizar un único servicio Docker porque el frontend se entrega desde FastAPI y no necesita una aplicación independiente.

5.1 Configurar el servicio

8. Iniciar sesión en https://dashboard.render.com/ y seleccionar New > Web Service.
9. Conectar GitHub y autorizar el acceso al repositorio riesgo-crediticio-fase-ll. Si es privado, conceder acceso explícito desde la integración de GitHub.
10. Elegir la rama main y seleccionar Docker como entorno de ejecución. Utilizar la raíz del repositorio como Root Directory y Dockerfile como archivo de construcción.
11. Asignar un nombre al servicio, elegir la región correspondiente y revisar los límites y costos del plan disponible. Para una demostración de corta duración, el plan Free puede ser suficiente si está disponible.
12. Configurar la comprobación de salud HTTP con la ruta /api/health. El Dockerfile utiliza Uvicorn en 0.0.0.0:8000; establecer PORT=8000 si el asistente de configuración lo solicita y comprobar que Render detecte el puerto.
13. Revisar la configuración, evitar publicar credenciales y crear el servicio. Render iniciará la construcción de la imagen y el despliegue.

Parámetro	Configuración propuesta
Servicio	Web Service
Repositorio	miguelalejandreperez-rgb/riesgo-crediticio-fase-ll
Rama	main
Runtime / Language	Docker
Dockerfile	./Dockerfile, situado en la raíz
Puerto del contenedor	8000, host 0.0.0.0
Health Check Path	/api/health
Modelo	Incluido en la imagen desde artifacts/
Credenciales	No incorporar tokens ni secretos en el repositorio

5.2 Verificar el despliegue

Una vez que Render indique el servicio como disponible, copiar la URL HTTPS asignada. Sustituir <servicio> por el subdominio real:

```
curl -i https://<servicio>.onrender.com/api/health
# Formulario: https://<servicio>.onrender.com/
# Swagger:   https://<servicio>.onrender.com/docs
```

Para comprobar la inferencia, abrir /docs > POST /api/predecir > Try it out, introducir los 19 valores de ejemplo del README y ejecutar. Una solicitud válida debe devolver HTTP 200; una inválida debe devolver HTTP 422 con la causa específica.

Estos pasos describen una verificación requerida para un despliegue futuro. No se dispone de una URL pública de Render ni de registros de ejecución en esa plataforma.

5.3 Disponibilidad y mantenimiento

- El modelo viaja dentro de la imagen Docker; no debe entrenarse durante cada despliegue ni en cada petición del formulario.
- Para publicar cambios, subir el código o una nueva versión de ambos artefactos a GitHub y efectuar un nuevo despliegue desde Render.
- Los servicios gratuitos de Render pueden entrar en reposo tras 15 minutos sin tráfico y reactivarse con demora; los archivos escritos localmente durante la ejecución no son persistentes.
- No compartir la URL públicamente ni procesar datos personales antes de implementar autenticación, controles de acceso, auditoría y validación externa del modelo.

6. Diagnóstico de incidencias

Síntoma	Comprobación y solución
Connection refused	Verificar docker ps. Ejecutar bash scripts/start_codespaces.sh y después smoke_docker.sh.
No such container	El daemon no conserva el contenedor. El script de inicio lo crea con el modelo versionado.
No module named joblib	Ocurre al intentar entrenar sin dependencias. Preparar el modelo solo si faltan los artefactos.
HTTP 404 del túnel	Abrir URL vigente desde Ports y confirmar la sesión de GitHub y visibilidad Private.
422 / Field required	Falta una variable obligatoria de la solicitud JSON.
422 / JSON decode error	Revisar comas, llaves, comillas y estructura JSON antes de evaluar valores extremos.
Render: build fallido	Verificar que artifacts/modelo_riesgo.joblib y artifacts/metadata.json estén en GitHub y disponibles en el contexto de Docker.
Render: error 502/health	Revisar registros, puerto, inicio Uvicorn y GET /api/health.

7. Criterios de cierre y trazabilidad

La implementación se considera verificable cuando existe un Dockerfile funcional, la aplicación devuelve HTTP 200 desde la API, el formulario muestra la misma puntuación que el endpoint para idénticos datos y el flujo de GitHub Actions concluye satisfactoriamente. Esas comprobaciones se documentan en Validacion_y_Pruebas.pdf.

Alcance comprobado Docker, FastAPI, formulario y GitHub Actions fueron probados en el entorno remoto de Codespaces. No se realizaron pruebas de Docker directamente en Windows ni un despliegue real en Render.

Referencias técnicas

- GitHub Docs. GitHub Codespaces: https://docs.github.com/en/codespaces
- GitHub Docs. Forwarding ports: https://docs.github.com/en/codespaces/developing-in-a-codespace/forwarding-ports-in-your-codespace
- Render. Docker on Render: https://render.com/docs/docker
- Render. Web Services: https://render.com/docs/web-services
- Render. Deploy for Free: https://render.com/docs/free
- Repositorio del proyecto: https://github.com/miguelalejandreperez-rgb/riesgo-crediticio-fase-ll